CMR UNIVERSITY



HD SUITE

Railway Reservation Form & Online Shopping Cart Simulation

A Java Swing Desktop Application

Java Mini Application Development - Project Report

Submitted by:	Hemashree T N Divya Reddy
USN:	25BBTCS093 25BBTCS067
Department:	B Tech CSE
Institution:	CMR University
Academic Year:	2nd Year

Abstract

HD Suite is a Java desktop application that brings together two GUI-based mini applications built with Swing and AWT: **HD Rail**, a railway reservation form, and **HD Cart**, an online shopping cart simulation. HD Rail lets a user search trains, view a detailed fare (base fare, concessions, reservation charge, GST and add-ons), book a ticket that is either confirmed with a berth or placed on a waiting list, and cancel it by PNR with an automatic refund and waiting-list promotion. HD Cart presents products as cards with pictures, ratings and discounts, and supports search, filtering, sorting, a cart with stock limits, coupons, gift wrap, payment modes and a three-step checkout that ends with an order receipt. The design follows object-oriented principles, with abstract Ticket, Product and Coupon classes whose subclasses supply their own rules, and the interface responds to users through the delegation event model. Every input is validated, and errors are reported through a custom checked exception. The build was verified with 98 automated logic checks and an automated walk through 31 screens of the running application.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope and Limitations
5. Tools and Technologies
6. System Design and Workflow
7. Calculations and Business Rules
8. Object-Oriented Design
9. Event Handling
10. Implementation and File Structure
11. User Interface and Screenshots
12. Testing and Observations
13. Conclusion and Future Enhancements
14. Source Code Appendix

1. Introduction

Many everyday tasks are now done through forms on a screen: booking a train ticket, ordering products online, paying for them. Behind each of these forms sits a program that must collect the user's input, check it, apply business rules such as fares, taxes, discounts and stock limits, and show a clear result. A small mistake in any of these steps, such as accepting an invalid date or charging the wrong amount, makes the whole application unreliable.

This project, **HD Suite**, is a desktop application written in Java using Swing and AWT. It contains two complete GUI applications that open from one home screen:

- **HD Rail - Railway Reservation Form (demo):** search trains between stations, see the fare breakup, book a ticket with berth allotment and a waiting list, and cancel by PNR with a refund.
- **HD Cart - Online Shopping Cart Simulation:** browse products with pictures, ratings and discounts, manage a cart, apply coupons, choose a payment mode and check out with an order receipt.

The application runs locally on any computer with a Java runtime. It needs no server, no database and no external library; even the product pictures are drawn in code with Java 2D. All trains, prices and ratings are demo data.

2. Problem Statement

To design and develop GUI-based mini applications in Java using Swing/AWT that simulate a railway reservation form and an online shopping cart, with a well laid-out interface, application logic built on core OOP concepts, event handling through the delegation event model, input validation with exception handling, and a working build that produces correct output for typical use cases.

3. Objectives

- Build two complete GUI applications with Java Swing/AWT, launched from a common home screen.
- Use a range of Swing components (labels, text fields, buttons, combo boxes, radio buttons, check boxes, menus, tables) arranged with suitable layout managers.
- Implement the application logic with classes, encapsulation, inheritance, abstraction and polymorphism.
- Respond to user actions with the delegation event model: ActionListener, ItemListener and adapter classes.
- Validate every input and handle errors and exceptions with clear messages.
- Deliver a build that compiles and runs without errors, and verify it with automated tests.

4. Scope and Limitations

Scope

HD Rail covers 16 trains between 6 cities, three travel classes (Sleeper, AC 3 Tier, AC 2 Tier), fare calculation with concessions and GST, berth allotment by preference, a waiting list of up to 5 per class, and cancellation with refund. HD Cart covers 13 products in 4 categories, search, filter and sort, a cart with quantity limits, three coupons, gift wrap, three payment modes, free-delivery rules and a simulated checkout with receipt.

Limitations

- Data is kept in memory only; bookings and orders are lost when the application closes.
- Trains, timings, fares, prices, ratings and tax rates are demo values, not live data.
- Payment is simulated; no money is charged and no payment details are collected.
- There are no user accounts or logins; the application serves one user at a time.
- Seat capacities are kept small (8, 6 and 4 berths) so that the waiting list is easy to demonstrate.

5. Tools and Technologies

Technology	Purpose
Java (JDK 17)	Programming language and runtime. Code is compiled with `--release 8` so it also runs on older lab computers.
Java Swing	GUI components: JFrame, JDialog, JPanel, JLabel, JTextField, JTextArea, JButton, JComboBox, JRadioButton, JCheckBox, JToggleButton, JTable, JProgressBar, JMenuBar, JOptionPane.
Java AWT	Layout managers (BorderLayout, GridLayout, FlowLayout, CardLayout, GridBagLayout), events and listener interfaces.
Java 2D (Graphics2D)	Drawing the product pictures, train and cart icons, star ratings and rounded buttons in code.
java.time	LocalDate and DateTimeFormatter to parse and check journey dates (dd-MM-yyyy).
java.util Collections	ArrayList, TreeSet and Comparator for bookings, products, the cart and sorting.
javac / java, run.bat	Compiling and running from the command line; run.bat does both in one click. Any editor such as VS Code can be used.

6. System Design and Workflow

The program is organised in three layers. Keeping the rules out of the GUI classes made it possible to test them on their own.

Layer	Classes	Responsibility
Presentation (GUI)	MainApp, RailwayReservationFrame, JourneyPanel, ShoppingCartFrame, ProductCard, CartRow, CheckoutDialog	Windows, components, layouts and event handling
Logic and model	ReservationSystem, Ticket and its 3 subclasses, Train, Passenger, Store, ShoppingCart, CartItem, Product and its 4 subclasses, Coupon	Fares, berths, waiting list, refunds, cart totals, coupons, stock
Common utilities	Validator, ValidationException, Money, Art, UiUtil, FlatButton	Input checks, rupee formatting, drawings, shared styling

HD Rail workflow

1. **Input:** the user enters passenger details and picks the From and To stations, train, class, date, berth preference and add-ons.
2. **Validation:** name, age, gender, mobile number, route and date are checked; any problem is shown in an error dialog.
3. **Fare:** *Check Fare* builds a ticket object of the chosen class and shows the full fare breakup.
4. **Booking:** after confirmation the system gives a PNR and either a confirmed berth (CNF) or a waiting-list number (WL).
5. **Cancellation:** the user enters a PNR; the refund is shown, and the first waiting ticket automatically gets the freed berth.

HD Cart workflow

1. **Browse:** products are filtered by category chip, live search and sort order.
2. **Cart:** items are added with a quantity; - / + steppers and Remove change the cart; stock limits are enforced.
3. **Options:** coupon, gift wrap and payment mode update the totals immediately.
4. **Checkout:** delivery details are validated inline, a payment step is simulated, then stock is reduced and a receipt is shown.

7. Calculations and Business Rules

Railway fare

```
Total fare = Base fare − Concession + Reservation charge + GST + Add-ons
Base fare = distance (km) × rate per km     GST is charged on (Base fare − Concession)
```

Class	Rate / km (Rs)	Reservation (Rs)	GST	Cancellation (Rs)	Berths
Sleeper (SL)	0.60	20	0%	120	8
AC 3 Tier (3A)	1.50	40	5%	180	6
AC 2 Tier (2A)	2.20	50	5%	200	4

Concession: senior citizens (60+) get 40% and children (5-11) get 50% off the base fare; children under 5 travel free and cannot book a berth. Meals cost Rs 150 (Rs 200 in AC 2 Tier) and insurance Rs 10. A waiting-list ticket is refunded less Rs 60. *Example:* Chennai Central to KSR Bengaluru, 362 km, AC 3 Tier, adult: 543.00 + 40.00 + 27.15 = **Rs 610.15**.

Shopping cart total

```
Grand total = Subtotal + Tax − Coupon discount + Delivery + Gift wrap / COD charges
```

Tax depends on the product category (Electronics 18%, Clothing 5% up to Rs 1,000 and 12% above, Books 0%, Grocery 5%; demo rates). Coupons: SAVE10 (10% off, maximum Rs 500), FLAT200 (Rs 200 off orders of Rs 1,500 or more) and FEST20 (20% off orders of Rs 3,000 or more, maximum Rs 1,000). Delivery is free from Rs 999, otherwise Rs 49; gift wrap adds Rs 50 and Cash on Delivery adds Rs 40. *Example:* 5,244.00 + 606.99 − 1,000.00 + 0 + 90.00 = **Rs 4,940.99**.

8. Object-Oriented Design

Concept	Where it is used
Classes and objects	Every real-world thing is a class: Train, Passenger, Ticket, Product, CartItem, ShoppingCart, Coupon, Store, ReservationSystem.
Encapsulation	All fields are private. State changes only through methods such as `confirm()`, `waitlist()`, `add()`, `updateQuantity()` and `applyCoupon()`, which check the rules first.
Inheritance	SleeperTicket, AcThreeTierTicket and AcTwoTierTicket extend Ticket; Electronics, Clothing, Book and Grocery extend Product; PercentCoupon and FlatCoupon extend Coupon; FlatButton extends JButton; the windows extend JFrame and JDialog.
Abstraction	Ticket, Product and Coupon are abstract classes. Abstract methods such as `getRatePerKm()`, `getGstPercent()`, `getTaxPercent()` and `discountOn()` force each subclass to supply its own rule.
Polymorphism	`Ticket.getTotalFare()` is written once and calls the overridden methods, so the same call gives the correct fare for any class. AcTwoTierTicket also overrides `getBerthType()` because AC 2 Tier has no middle berth. `CartItem.getTax()` works the same way for every product category.

9. Event Handling

The application uses the **delegation event model**: a component (the event source) creates an event object and delivers it to the listener objects registered with it. Each window implements the listener interfaces it needs and checks `getSource()` or `getActionCommand()` to decide what to do.

Listener	Used for
ActionListener	All buttons and menu items, pressing Enter in text fields, and the custom ActionEvents sent by ProductCard and CartRow.
ItemListener	Station, train and class combo boxes; category chips and sort order; gift-wrap check box; payment radio buttons.
MouseAdapter	Highlighting cards on hover, opening product details by clicking the picture, double-clicking a booking to view its ticket.
KeyAdapter	Allowing only digits in age, mobile, quantity and PIN code fields; live product search while typing.
FocusAdapter	Re-checking seat availability when the user leaves the journey-date field.
WindowAdapter	Asking for confirmation before closing the application.
javax.swing.Timer	The live clock on the home screen, short notification messages and the payment animation.

10. Implementation and File Structure

The project folder contains `src/` (the 29 Java source files, 4,859 lines in total), `out/` (compiled .class files, created by javac), `screenshots/`, `run.bat` and `README.md`. To run it: `javac -encoding UTF-8 -d out src\*.java` followed by `java -cp out MainApp`, or simply double-click `run.bat`.

File(s)	Description
MainApp.java	Program entry point and home screen with the two module cards, live clock and session counters.
RailwayReservationFrame.java, JourneyPanel.java	HD Rail window: passenger form, journey strip, e-ticket preview and bookings table.
ReservationSystem.java	Trains, bookings, PNR numbers, berth allotment, waiting list and cancellation.
Train.java, Passenger.java	Model classes for a train and a passenger.
Ticket.java + SleeperTicket, AcThreeTierTicket, AcTwoTierTicket	Abstract ticket with the fare formula and e-ticket text; one subclass per travel class.
ShoppingCartFrame.java, ProductCard.java, CartRow.java	HD Cart window: top bar, category chips, product grid and cart sidebar.

File(s)	Description
CheckoutDialog.java	Three-step checkout (details, payment, confirmation) using CardLayout.
Store.java, ShoppingCart.java, CartItem.java, Coupon.java	Catalogue, orders and receipt; cart rules and totals; coupon types.
Product.java + Electronics, Clothing, Book, Grocery	Abstract product with listing details; one subclass per category.
Validator.java, ValidationException.java	Shared input checks and the custom checked exception.
Art.java, FlatButton.java, UiUtil.java, Money.java	Java 2D drawings, rounded buttons, shared styling, rupee formatting.

11. User Interface and Screenshots

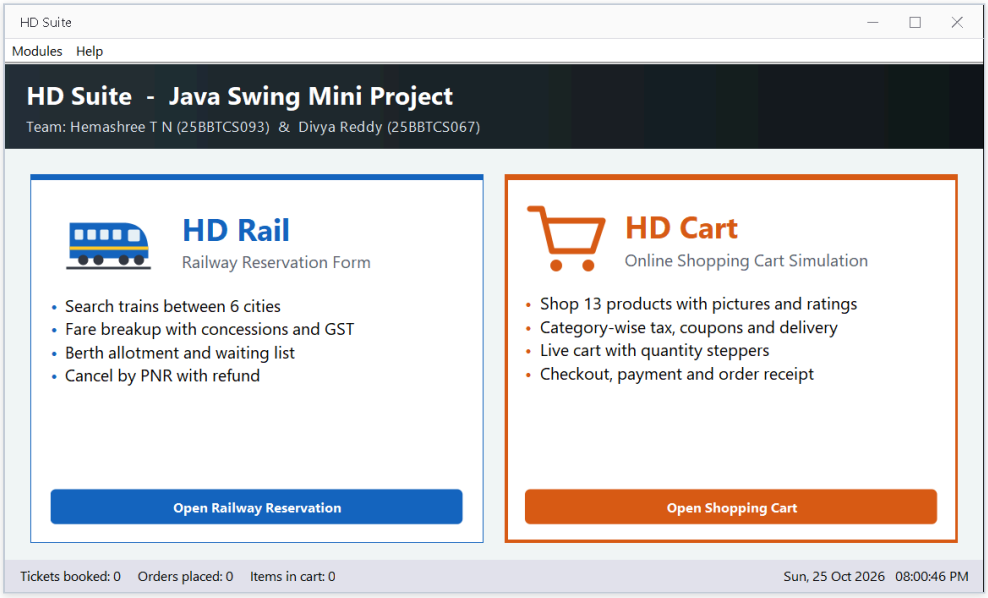


Figure 1. HD Suite home screen with both modules, the team details and the live status bar.

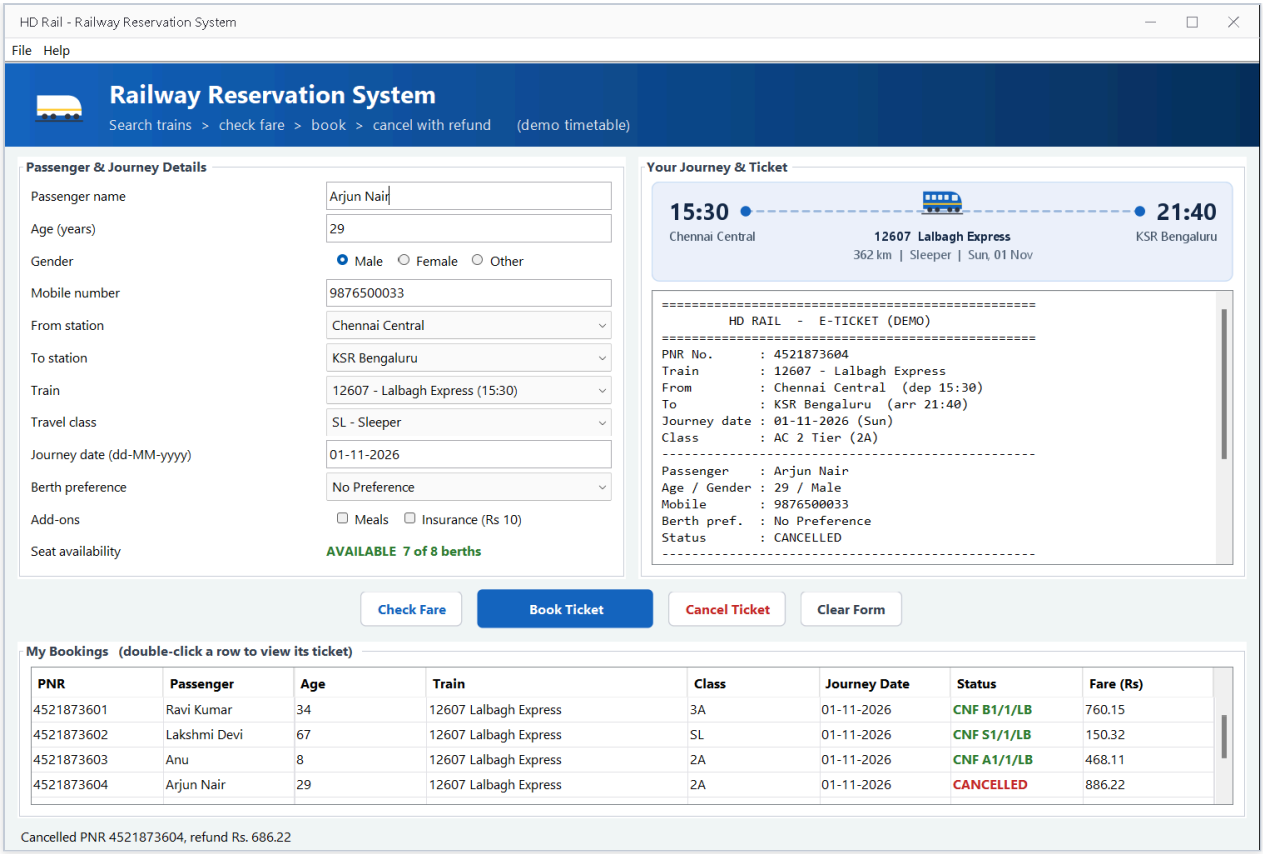


Figure 2. HD Rail after a cancellation: the form, journey strip, e-ticket with refund and the colour-coded bookings table.

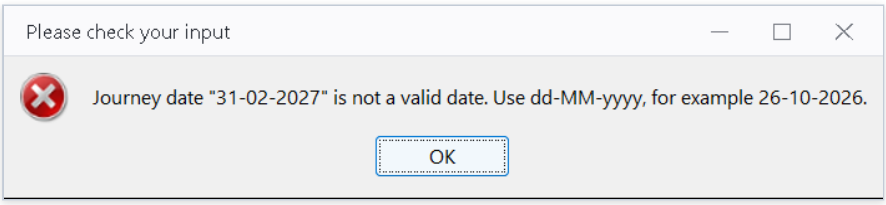


Figure 3. Validation: an impossible date (31-02-2027) is rejected with a clear message.

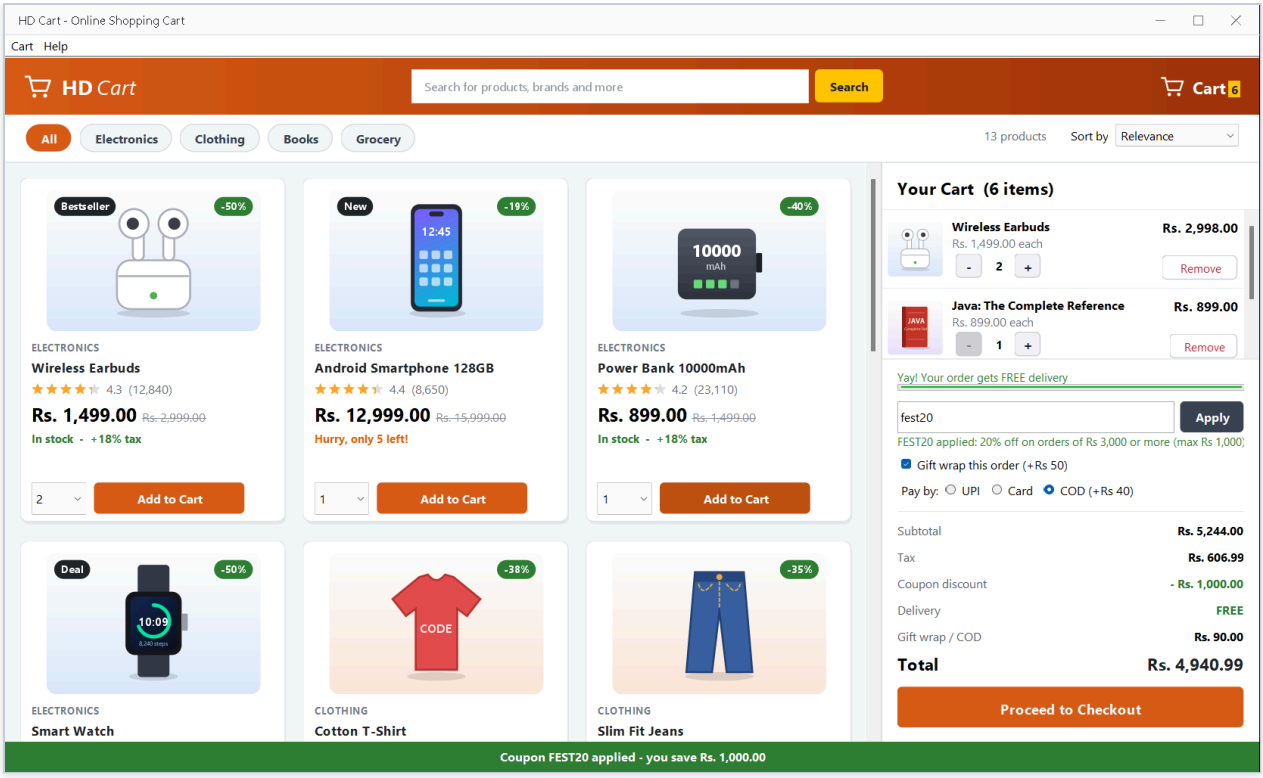


Figure 4. HD Cart: product cards with ratings and discounts, and the cart with coupon FEST20, gift wrap and Cash on Delivery.

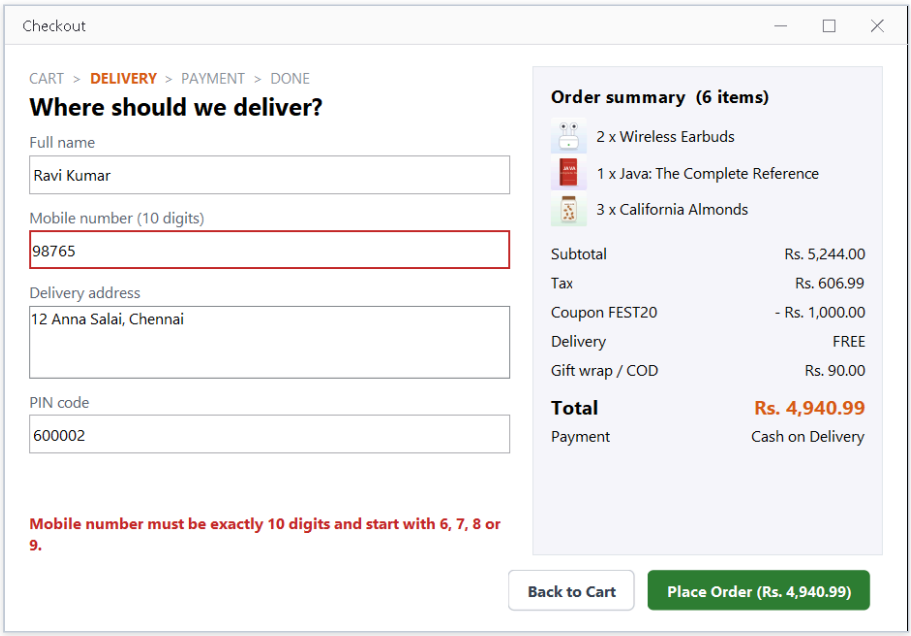


Figure 5. Checkout step 1: an invalid mobile number is outlined in red with an inline message.

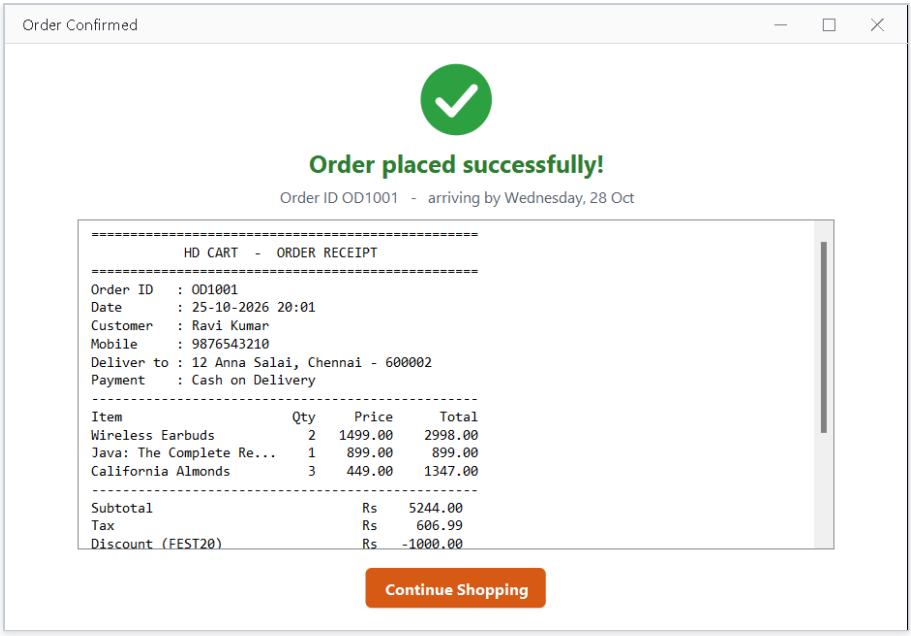


Figure 6. Checkout step 3: order confirmed, with the order ID, delivery date and the full receipt.

12. Testing and Observations

Testing was done at two levels. A logic test program checked the model classes with exact expected values (**98 checks, all passed**). An automated GUI test then drove the real windows of both modules, clicking buttons, filling forms and answering dialogs, and captured **31 screens** with **0 failures** in the final run.

Test case	Expected behaviour	Result
Book with an empty name	Error dialog; nothing is booked	**Pass**
Age typed as letters	Letters are blocked while typing	**Pass**
Journey date 31-02-2027	"not a valid date" message	**Pass**
Journey date in the past / beyond 120 days	Matching error message	**Pass**
Mobile number 12345	"exactly 10 digits" message	**Pass**
From and To stations the same	No trains listed; warning shown	**Pass**
AC 3 Tier, adult, Chennai to Bengaluru	Fare Rs 610.15	**Pass**
Senior citizen, no berth preference	Lower berth allotted, 40% concession	**Pass**
5th booking in a full AC 2 Tier	Ticket goes to WL 1	**Pass**
Booking when the waiting list is full	REGRET message	**Pass**
Same passenger booked twice on one train	Duplicate booking refused	**Pass**

Test case	Expected behaviour	Result
Cancel a confirmed AC 2 Tier ticket	Refund = fare - Rs 200; WL 1 confirmed	Pass
Cancel with a 5-digit PNR	"exactly 10 digits" message	Pass
Add more items than in stock	Stock limit message	Pass
Invalid coupon HELLO	"not valid" message	Pass
FEST20 on a Rs 5,244 cart	Discount capped at Rs 1,000	Pass
Coupon kept after removing items	Discount drops to 0 below the minimum	Pass
Checkout with mobile 98765	Red inline error; order not placed	Pass
Valid checkout (COD, gift wrap)	Receipt total Rs 4,940.99; stock reduced	Pass
Close and reopen a module window	Bookings and cart are kept	Pass

Defects found and fixed during testing: the bookings table could be squeezed to zero height on smaller screens; the add-ons and passenger details carried over to the next booking; and the payment progress bar was stretched to fill the dialog. All three were fixed and re-tested.

13. Conclusion and Future Enhancements

HD Suite meets all the requirements of the experiment. It has a well laid-out Swing interface built with several layout managers; its logic uses classes, encapsulation, inheritance, abstraction and polymorphism; it responds to user actions through the delegation event model with listeners and adapter classes; every input is validated, with exceptions handled and explained to the user; and the build compiles and runs without errors, producing correct results that were verified by automated tests.

Future enhancements

- Store bookings, products and orders in a database using JDBC and MySQL, so data survives a restart.
- Add user accounts with login and a history of bookings and orders.
- Export e-tickets and order receipts as PDF files.
- Show a visual seat map so that passengers can pick a berth.
- Connect to a real payment gateway in sandbox (test) mode.

14. Source Code Appendix

The listings below are copied directly from the project's source files. The complete source code of all 29 files is in the `src` folder.

A.1 Ticket.java - abstract rules and the fare calculation (polymorphism)

```
// ---------- rules that every travel class must define ----------

public abstract String getClassCode();

public abstract String getClassName();

/** Number of berths in this class on one train (kept small so the waiting list is easy to demo).
*/
public abstract int getTotalSeats();

protected abstract double getRatePerKm();

protected abstract double getReservationCharge();

protected abstract double getGstPercent();

protected abstract String getCoachCode();

public abstract double getCancellationCharge();

// ---------- rules a subclass MAY override ----------

protected double getMealPrice() {
    return 150.0;
}

/** Sleeper / AC 3 Tier bay: Lower, Middle, Upper x2, then Side Lower, Side Upper. */
public String getBerthType(int seat) {
    String[] bay = {"LB", "MB", "UB", "LB", "MB", "UB", "SL", "SU"};
    return bay[(seat - 1) % bay.length];
}

// ---------- fare calculation (same for every class) ----------

public double getBaseFare() {
    return Money.round(train.getDistanceKm() * getRatePerKm());
}

public int getConcessionPercent() {
    if (passenger.isSeniorCitizen()) {
        return 40;
    }
    if (passenger.isChild()) {
        return 50;
    }
    return 0;
}

public double getConcession() {
    return Money.round(getBaseFare() * getConcessionPercent() / 100.0);
}

public double getGst() {
```

```
    return Money.round((getBaseFare() - getConcession()) * getGstPercent() / 100.0);
}

public double getAddOnCharges() {
    double total = 0;
    if (mealsIncluded) {
        total += getMealPrice();
    }
    if (insuranceIncluded) {
        total += INSURANCE_PREMIUM;
    }
    return total;
}

public double getTotalFare() {
    return Money.round(getBaseFare() - getConcession() + getReservationCharge() + getGst() +
getAddOnCharges());
}
```

A.2 AcTwoTierTicket.java - a subclass overriding the rules

```
import java.time.LocalDate;

/**
 * AC 2 Tier (2A): has no middle berth, so it overrides the berth layout,
 * and it serves a costlier meal.
 */
public class AcTwoTierTicket extends Ticket {

    public static final int SEATS = 4;

    public AcTwoTierTicket(String pnr, Passenger passenger, Train train, LocalDate journeyDate,
                           String berthPreference, boolean meals, boolean insurance) {
        super(pnr, passenger, train, journeyDate, berthPreference, meals, insurance);
    }

    @Override
    public String getClassCode() {
        return "2A";
    }

    @Override
    public String getClassName() {
        return "AC 2 Tier";
    }

    @Override
    public int getTotalSeats() {
        return SEATS;
    }

    @Override
    protected double getRatePerKm() {
        return 2.20;
    }

    @Override
    protected double getReservationCharge() {
        return 50.0;
    }

    @Override
    protected double getGstPercent() {
```

```
        return 5.0;
    }

    @Override
    protected String getCoachCode() {
        return "A1";
    }

    @Override
    public double getCancellationCharge() {
        return 200.0;
    }

    @Override
    protected double getMealPrice() {
        return 200.0;
    }

    /** 2A bay: Lower, Upper x2, then Side Lower, Side Upper (no middle berth). */
    @Override
    public String getBerthType(int seat) {
        String[] bay = {"LB", "UB", "LB", "UB", "SL", "SU"};
        return bay[(seat - 1) % bay.length];
    }
}
```

A.3 ReservationSystem.java - booking and cancellation with the waiting list

```
public Ticket book(String classCode, Passenger passenger, Train train, LocalDate date,
                   String berth, boolean meals, boolean insurance) throws ValidationException {
    for (Ticket existing : bookings) {
        if (existing.getStatus() != Ticket.Status.CANCELLED
                && existing.getTrain() == train
                && existing.getJourneyDate().equals(date)
                && existing.getPassenger().getName().equalsIgnoreCase(passenger.getName())
                && existing.getPassenger().getMobile().equals(passenger.getMobile())) {
            throw new ValidationException(passenger.getName() + " already has a booking on this
train and date (PNR "
                    + existing.getPnr() + ").");
        }
    }

    Ticket ticket = createTicket(String.valueOf(nextPnr), classCode, passenger, train, date,
berth, meals, insurance);
    int seat = pickSeat(ticket);
    if (seat > 0) {
        ticket.confirm(seat);
    } else {
        int waiting = getWaitingCount(train, classCode, date);
        if (waiting >= MAX_WAITING_LIST) {
            throw new ValidationException("REGRET: " + ticket.getClassName() + " on train " +
train.getNumber()
                    + " is full and the waiting list is closed. Please try another class, train or
date.");
        }
        ticket.waitlist(waiting + 1);
    }
    nextPnr++;
    bookings.add(ticket);
    return ticket;
}

/** Cancels a ticket, works out the refund and moves the waiting list up. */
```

```
public Ticket cancel(String pnrInput) throws ValidationException {
    Ticket ticket = requireActiveTicket(pnrInput);
    boolean wasConfirmed = ticket.getStatus() == Ticket.Status.CONFIRMED;
    int freedSeat = ticket.getSeatNumber();
    double charge = wasConfirmed ? ticket.getCancellationCharge() : WAITLIST_CANCELLATION_CHARGE;
    ticket.cancel(Money.round(Math.max(0, ticket.getTotalFare() - charge)));

    List<Ticket> waiting = waitingList(ticket.getTrain(), ticket.getClassCode(),
ticket.getJourneyDate());
    if (wasConfirmed && !waiting.isEmpty()) {
        waiting.remove(0).confirm(freedSeat);   // WL 1 gets the freed berth
    }
    for (int i = 0; i < waiting.size(); i++) {
        waiting.get(i).waitlist(i + 1);         // everyone else moves up
    }
    return ticket;
}
```

A.4 Validator.java - input checks with exception handling

```
public static int parseIntInRange(String raw, String field, int min, int max) throws
ValidationException {
    String text = requireText(raw, field, 1);
    int value;
    try {
        value = Integer.parseInt(text);
    } catch (NumberFormatException e) {
        throw new ValidationException(field + " must be a valid whole number (you entered \"" +
text + "\").");
    }
    if (value < min || value > max) {
        throw new ValidationException(field + " must be between " + min + " and " + max + ".");
    }
    return value;
}

/** Parses a dd-MM-yyyy date that must lie between today and today + maxDaysAhead. */
public static LocalDate parseFutureDate(String raw, String field, LocalDate today, int
maxDaysAhead)
        throws ValidationException {
    String text = requireText(raw, field, 1);
    LocalDate date;
    try {
        date = LocalDate.parse(text, DATE_FORMAT);
    } catch (DateTimeParseException e) {
        throw new ValidationException(field + " \"" + text + "\" is not a valid date. Use dd-MM-
yyyy, for example "
                + today.plusDays(1).format(DATE_FORMAT) + ".");
    }
    if (date.isBefore(today)) {
        throw new ValidationException(field + " cannot be in the past.");
    }
    LocalDate lastDate = today.plusDays(maxDaysAhead);
    if (date.isAfter(lastDate)) {
        throw new ValidationException("Bookings open only " + maxDaysAhead + " days in advance (up
to "
                + lastDate.format(DATE_FORMAT) + ").");
    }
    return date;
}
```

A.5 ShoppingCart.java - adding items and computing the totals

```
public void add(Product product, int quantity) throws ValidationException {
    if (product == null) {
        throw new ValidationException("Please select a product first.");
    }
    if (quantity < 1 || quantity > MAX_QUANTITY_PER_ADD) {
        throw new ValidationException("Quantity must be between 1 and " + MAX_QUANTITY_PER_ADD +
".");
    }
    if (product.getStock() == 0) {
        throw new ValidationException(product.getName() + " is out of stock.");
    }
    CartItem existing = find(product);
    int already = existing == null ? 0 : existing.getQuantity();
    if (already + quantity > product.getStock()) {
        String message = "Only " + product.getStock() + " unit(s) of " + product.getName() + " are
in stock";
        message += already > 0 ? " and you already have " + already + " in your cart." : ".";
        throw new ValidationException(message);
    }
    if (existing == null) {
        items.add(new CartItem(product, quantity));
    } else {
        existing.setQuantity(already + quantity);
    }
}

/** 0 if there is no coupon, or if items were removed and the coupon no longer qualifies. */
public double getDiscount() {
    if (coupon == null || !coupon.isEligible(getSubtotal())) {
        return 0;
    }
    return coupon.discountOn(getSubtotal());
}

public double getGrandTotal() {
    return Money.round(getSubtotal() + getTax() - getDiscount() + getDeliveryCharge() +
getOtherCharges());
}
```

A.6 RailwayReservationFrame.java - registering listeners (delegation event model)

```
private void registerListeners() {
    AbstractButton[] actionSources = {fareButton, bookButton, cancelButton, clearButton,
        newItem, cancelItem, closeItem, rulesItem, aboutItem};
    for (AbstractButton source : actionSources) {
        source.addActionListener(this);
    }
    dateField.addActionListener(this);

    fromCombo.addItemListener(this);
    toCombo.addItemListener(this);
    trainCombo.addItemListener(this);
    classCombo.addItemListener(this);

    // FocusAdapter: re-check seats when the user leaves the date field
    dateField.addFocusListener(new FocusAdapter() {
        @Override
        public void focusLost(FocusEvent e) {
            refreshAvailability();
        }
```

```
    });

    // MouseAdapter: double-click a booking to see its ticket
    bookingTable.addMouseListener(new MouseAdapter() {
        @Override
        public void mouseClicked(MouseEvent e) {
            if (e.getClickCount() == 2) {
                showSelectedBooking();
            }
        }
    });

    // KeyAdapter: digits only
    UiUtil.allowDigitsOnly(ageField, 3);
    UiUtil.allowDigitsOnly(mobileField, 10);
}

@Override
public void actionPerformed(ActionEvent e) {
    Object source = e.getSource();
    if (source == fareButton) {
        checkFare();
    } else if (source == bookButton) {
        bookTicket();
    } else if (source == cancelButton || source == cancelItem) {
        cancelTicket();
    } else if (source == clearButton || source == newItem) {
        clearForm();
    } else if (source == dateField) {
        refreshAvailability();
    } else if (source == rulesItem) {
        showFareRules();
    } else if (source == aboutItem) {
        showAbout();
    } else if (source == closeItem) {
        dispose();
    }
}
```